SOPORTE INNCENTIVES · DOCUMENTACIÓN INTERNA

Manual técnico

Arquitectura, infraestructura, despliegue, seguridad y operación de la mesa de soporte de Inncentives, para el equipo técnico de Neitek.

Para: equipo técnico de Neitek **Versión:** 1.0 · octubre 2026 **Clasificación:** uso interno

01 Resumen

Qué es la mesa de soporte, cómo está construida y dónde vive cada parte.

La mesa de soporte de Inncentives es un help desk inspirado en Zammad para los clientes de Inncentives. Tiene dos áreas que se distinguen por el host: el **Portal de clientes** y el **área de agentes**; cada host tiene su propia cookie de sesión y las rutas de administración solo responden en el host de agentes.

ÁREA	HOST EN PRODUCCIÓN	NOTAS
Portal de clientes	`supportinncentives.neitek.net`	También publica `/hooks/*` (proveedores) y `/widget.js`.
Agentes	`agents-supportinncentives.neitek.net`	Rutas de administración y tokens de API solo aquí.

NOMBRES DE DOMINIO EN LA DOCUMENTACIÓN

La guía de despliegue y algunas integraciones todavía usan los nombres `soporte.inncentives.com` y `agentes.soporte.inncentives.com`. Producción usa los hosts de `neitek.net` de la tabla, configurados en `public_hosts` de Terraform.

Tecnología

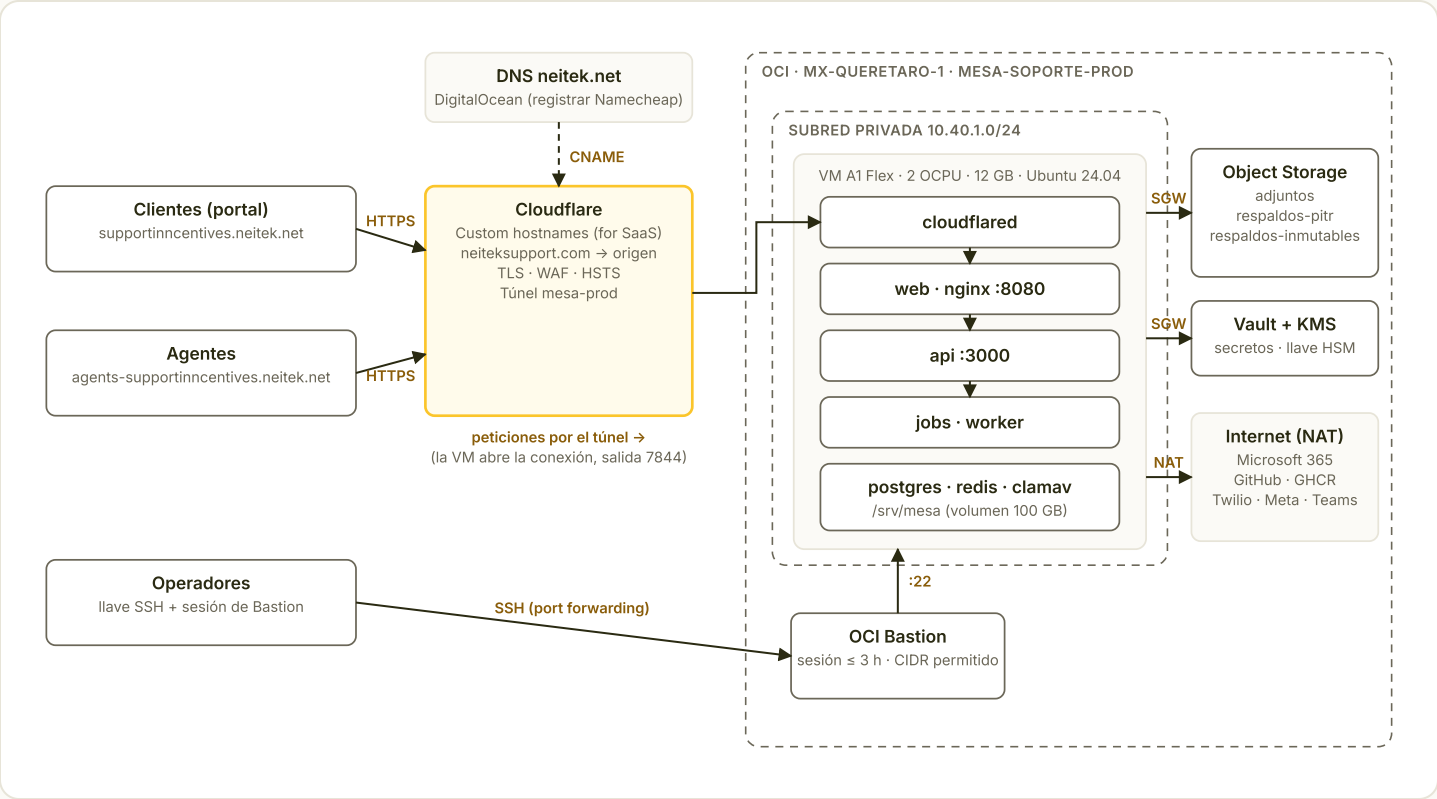
CAPA	TECNOLOGÍA
Lenguaje	TypeScript 5.9 sobre Node.js 22 (pnpm 10, monorepo)
API	NestJS 11 sobre Fastify 5, validación con Zod, Prisma 6
Datos	PostgreSQL 16 (imagen propia con pgBackRest y age), Redis 7.4
Procesos en segundo plano	BullMQ 5 (colas en Redis)
Tiempo real	Socket.IO 4.8 solo por WebSocket
Web	React 18, Vite 7, Tailwind 3, TanStack Query y Router, i18next (es/en)
Servidor web	nginx-unprivileged 1.30 (sustituyó a Caddy, ADR-0001)
Imágenes	Distroless Node 22 (nonroot) fijada por digest; multi-arquitectura amd64/arm64
Pruebas	Vitest, Playwright, OWASP ZAP (DAST), pruebas de respaldo y desplegador en shell

Estructura del repositorio

RUTA	CONTENIDO
`packages/shared`	Esquemas Zod, permisos por rol, catálogo de condiciones de reglas, nombres de colas, tipos de la API.
`apps/api`	API NestJS; el mismo código genera el proceso `jobs`. Esquema y migraciones de Prisma.
`apps/worker`	Worker BullMQ del correo del sistema.
`apps/web`	SPA de agentes y portal.
`docker/`	`app.Dockerfile` multi-target (api, migrator, worker, web), imagen de PostgreSQL, configuración de nginx.
`deploy/`	Compose de producción, scripts del servidor (desplegador, respaldos, secretos, métricas) y unidades systemd.
`infra/`	Terraform de OCI, cloud-init y `bootstrap-secrets.sh`.
`docs/`	Guía de despliegue, ADRs, seguridad, integraciones, runbooks y manuales.

02 Topología y red

Cómo llega una petición desde el navegador hasta el contenedor que la atiende.



Topología. No hay puertos abiertos hacia la VM: el túnel lo inicia `cloudflared` desde adentro. Object Storage y Vault se alcanzan por el Service Gateway; el resto de la salida (Microsoft 365, GitHub, proveedores) por NAT.

Borde: Cloudflare

- **DNS.** La zona `neitek.net` sigue en DigitalOcean (registrar Namecheap). Los dos hosts públicos son CNAME hacia Cloudflare for SaaS, configurado sobre el dominio técnico `neiteksupport.com` con origen de respaldo `origen.neiteksupport.com`. Los registros TXT de validación también viven en DigitalOcean.
- **Túnel** `mesa-prod`: entrega ambos hosts a `http://web:8080`. TLS termina en Cloudflare; dentro del túnel el tráfico llega en HTTP a nginx.
- **IP del cliente.** La API solo confía en `CF-Connecting-IP` cuando la petición viene de nginx (`172.30.15.10`).
- **Controles previstos** (guía de despliegue §8): reglas administradas del WAF, Bot Fight Mode en el portal, regla *Skip* para `/hooks/`, límite de 20 peticiones por minuto a `/api/v1/auth/*`, Always HTTPS, TLS mínimo 1.2 y HSTS, y Cloudflare Access para el host de agentes (grupos de Entra con MFA, *Bypass* en `/hooks/*`).

Red en OCI

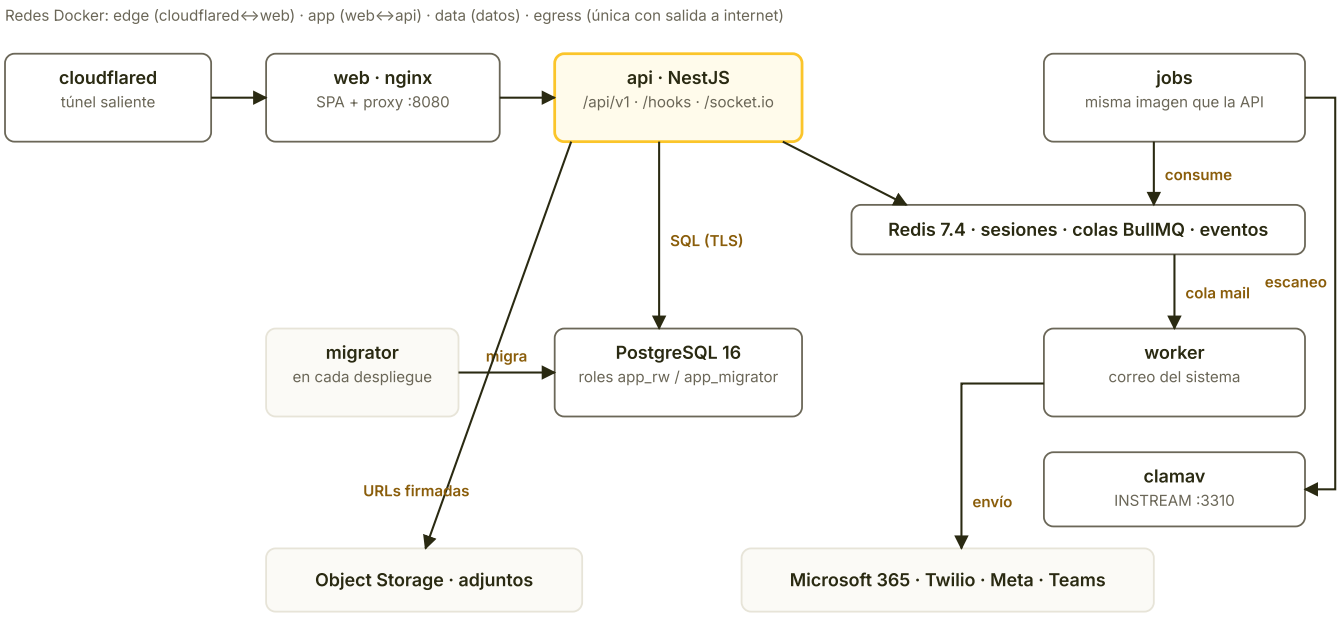
ELEMENTO	CONFIGURACIÓN
VCN	`10.40.0.0/16`; subred privada `10.40.1.0/24` sin IP pública ni entrada desde internet.
Salida	NAT Gateway para internet; Service Gateway para los servicios de Oracle (Object Storage, Vault).
Security list	Se reemplaza la predeterminada por `mesa-prod-default-deny`. Su única regla es la salida TCP 22 hacia la subred, necesaria para que el endpoint de Bastion alcance la VM.
NSG `mesa-prod-app-nsg`	Entrada: solo TCP 22 desde la subred (Bastion). Salida: 443, 80 (apt), 7844 TCP/UDP (túnel), 587, 993, 53/udp, 123/udp y servicios de Oracle.
Bastion	`mesa-prod-bastion`, lista blanca de CIDR (nunca `0.0.0.0/0`), sesiones de hasta 3 h.

ACCESO DE OPERADORES

Se entra con una sesión de **port forwarding** de Bastion hacia el puerto 22 de la VM (por ejemplo `ssh -N -L 2222:<ip>:22 <sesión>@host.bastion.mx-queretaro-1.oci.oraclecloud.com`) y luego `ssh -p 2222 ubuntu@localhost`. Las llaves públicas autorizadas se instalan desde `admin_ssh_public_keys` de Terraform.

03 Contenedores

La aplicación corre como una pila de Docker Compose, proyecto `mesa`.



Contenedores. Una misma imagen corre dos procesos: `api` (HTTP) y `jobs` (colas y tareas periódicas); ambos leen y escriben en PostgreSQL. Redis es el punto de encuentro: sesiones, colas BullMQ y el canal de eventos en tiempo real.

SERVICIO	IMAGEN	FUNCIÓN	LÍMITES
cloudflared	`cloudflare/cloudflared:2025.10.0`	Túnel saliente; token desde archivo.	256 MB
web	mesa-web (nginx)	Sirve la SPA y hace proxy de `/api`, `/hooks` y `/socket.io`.	256 MB
api	mesa-api	API HTTP en :3000; healthcheck a `/health`.	1 GB
jobs	mesa-api (`dist/jobs-main.js`)	Colas y tareas periódicas, sin HTTP.	1 GB
worker	mesa-worker	Correo del sistema por SMTP (Office 365) o Graph.	512 MB
migrator	mesa-migrator	`prisma migrate deploy`; perfil `migrate`, solo lo invoca el desplegador.	—
postgres	mesa-postgres	PostgreSQL 16 con pgBackRest; datos en `/srv/mesa/pgdata`.	3 GB
redis	`redis:7.4-alpine`	Sesiones, colas, limitadores, pub/sub; AOF; `maxmemory 384mb noeviction`.	512 MB
clamav	`clamav/clamav-debian:1.4`	Antivirus INSTREAM en :3310; perfil `default`.	3 GB

Redes Docker

RED	SUBRED	USO
edge	172.30.10.0/24 (interna)	cloudflared ↔ web
app	172.30.15.0/24 (interna)	web ↔ api; nginx tiene la IP fija 172.30.15.10
data	172.30.20.0/24 (interna)	Servicios de datos
egress	172.30.30.0/24	Única red con salida: Vault, Object Storage, Microsoft 365, Cloudflare, firmas de ClamAV

Endurecimiento común

- Sistema de archivos de solo lectura, `no-new-privileges`, `cap_drop: ALL`, usuarios no root y límites de memoria y procesos.
- Logs JSON rotados (20 MB × 5).
- Secretos como archivos: el tmpfs del host `/run/mesa/secrets` se monta en cada contenedor como `/run/secrets`. En producción la aplicación rechaza secretos por variable de entorno.
- Imágenes propias fijadas por digest en `/opt/mesa/images.env`; las de terceros (cloudflared, redis, clamav) por tag.

04 La aplicación

Cómo está organizada la API y qué mecanismos protegen cada petición.

Arranque

Fastify con límite de cuerpo de 1 MB, `trustProxy` y helmet. Prefijo global `/api/v1`, salvo `/health`, `/ready` y los webhooks de proveedores (`/hooks/cti`, `/hooks/azure-devops`, `/hooks/twilio/*`, `/hooks/whatsapp`). Respuestas con `Cache-Control: no-store`. La especificación OpenAPI está en `/api/v1/openapi.json` (requiere sesión de agente).

Módulos de la API

- **auth**: login, MFA TOTP, WebAuthn, SSO OIDC.
- **policy**: guardia global de autorización.
- **security**: contraseñas, sesiones, tokens, cifrado de campos, rate limit.
- **tickets**: tickets, artículos, macros, numeración.
- **attachments** y **storage**: adjuntos en OCI u S3.
- **automation**, **rules**, **sla**: triggers, SLA, Core Workflows.
- **overviews** y **search**: vistas y búsqueda.
- **channels**: correo, teléfono, WhatsApp/SMS, Twilio.
- **chat**: chat en vivo.
- **kb**: base de conocimiento bilingüe.
- **notifications** y **mail**: avisos in-app y correo.
- **realtime** y **events**: Socket.IO y bus de eventos.
- **reports**: métricas y exportación.
- **integrations**: Inncentives, Azure DevOps, Teams.
- **ai**: asistente para agentes.
- **privacy**: ARCO, retención, datos sensibles.
- **audit**: bitácora encadenada.
- **admin** y **config-admin**: usuarios, organizaciones, configuración, tokens, ver como cliente.
- **jobs** y **queue**: procesos en segundo plano.
- **health**, **settings**, **openapi**, **common**.

Sesiones y autenticación

MECANISMO	DETALLE
Cookie	`__Host-sid` de 256 bits, Secure, HttpOnly, SameSite=Lax, una por host. Redis guarda solo el SHA-256.
Expiración	30 min de inactividad, 12 h absoluta; el identificador rota al completar MFA y al cambiar la contraseña.
CSRF	Origin/Referer exacto + `Sec-Fetch-Site`, y token `X-CSRF-Token` por sesión.
Contraseñas	Argon2id (19 MiB, t=2), mínimo 12 caracteres, verificación de filtraciones (HIBP, k-anonymity).
MFA	Obligatorio para ADMIN y AGENT. TOTP con anti-replay y semilla cifrada; WebAuthn/passkeys; códigos de recuperación.
SSO OIDC	Code + PKCE; solo identidades vinculadas por el propio usuario; Entra ID para agentes.
Reautenticación	Acciones sensibles piden contraseña de nuevo (`@RequireReauth`).
Tokens de API	Personales, solo personal y host de agentes; se guarda el hash; vigencia ≤ 365 días.

Autorización

Denegar por defecto: cada ruta declara `@Public()` o `@RequirePermissions()`. Los permisos salen del rol (ADMIN, AGENT, CUSTOMER) más roles personalizados. El alcance por objeto lo da el acceso a grupos del agente o la organización del cliente; lo que queda fuera de alcance responde 404.

Otros mecanismos

- **Límites de frecuencia** en Redis: por ejemplo 5 intentos de login por cuenta cada 15 min, 30 por IP, y límites de subida, descarga y IA.
- **Bitácora**: cada registro guarda el hash del anterior (SHA-256); un trigger de la base impide modificar o borrar, y el rol de la aplicación no tiene esos permisos.

- **Tiempo real**: Socket.IO autenticado con la misma cookie; los eventos son mínimos (`{type, ticketId}`) y viajan por el canal Redis `mesa:events`.
- **Búsqueda**: texto completo de PostgreSQL (columna generada con índice GIN, español, inglés y sin acentos).
- **Reglas como datos**: condiciones JSON validadas contra un catálogo cerrado (máx. 60 nodos, 4 niveles); control de bucles en triggers.
- **Adjuntos**: lista blanca de tipos con verificación de bytes reales, 25 MB, escaneo con ClamAV antes de permitir descarga, URL firmada de 60 s, vistas previas recodificadas sin EXIF.
- **Datos sensibles**: detector de RFC, CURP, NSS, CLABE, tarjetas y sueldos en texto y archivos; aviso al cliente y redacción por el agente.

Canales e integraciones

INTEGRACIÓN	CÓMO FUNCIONA
Correo entrante	Microsoft Graph o IMAP cada minuto. Solo se confía en `Authentication-Results` del servidor configurado; sin *pass* el ticket va a «Por verificar». Hilos por cabeceras y por `[Ticket#…]`.
Correo saliente	Respuestas de tickets: cola `outbound` en `jobs`. Correo del sistema: cola `mail` en `worker` (SMTP STARTTLS u Graph).
Teléfono, WhatsApp, SMS	Webhooks firmados (Twilio, Meta, CTI con HMAC y anti-replay); deduplicación por id externo.
Chat en vivo	Cada chat es un ticket del canal chat; latido de disponibilidad de 90 s; cierre por inactividad.
Inncentives	Barra lateral (modo off/mock/http, caché 60 s) y widget `/widget.js` con JWT de un solo uso.
Azure DevOps, Teams, webhooks	Work items sincronizados cada 15 min; avisos a Teams; webhooks con protección SSRF y registro de entregas.
IA	Desactivada por defecto en dos niveles; redacción previa de datos; nunca envía adjuntos; solo sugiere.

05 Procesos en segundo plano

Lo que no ocurre dentro de una petición HTTP corre en `jobs` o en `worker`.

COLA	PROCESO	QUÉ HACE
`outbound`	jobs	Envía cada respuesta por correo, WhatsApp o SMS.
`webhooks`	jobs	Webhooks de triggers y avisos a Teams (5 intentos).
`scan`	jobs	Antivirus y datos sensibles de adjuntos nuevos.
`maintenance`	jobs	Tareas periódicas (tabla siguiente).
`mail`	worker	Invitaciones, restablecimiento de contraseña y notificaciones.

TAREA PERIÓDICA	FRECUENCIA
Correo entrante, SLA, pendientes, planificadores, privacidad, cierre de chats inactivos	cada minuto
Sincronización con Azure DevOps	cada 15 min
Limpieza (adjuntos sin usar, etc.)	cada hora
Retención de datos	cada 6 h

Los trabajos tienen 8 intentos con espera exponencial (desde 30 s); los completados se borran a las 24 h y los fallidos a los 7 días.

06 Datos

PostgreSQL es la fuente de verdad; Redis guarda estado efímero y colas.

DOMINIO	MODELOS PRINCIPALES
Identidad y acceso	Organization, User, Invitation, CustomRole, UserIdentity (SSO), WebAuthnCredential, ApiToken, RecoveryCode
Soporte	Group, GroupAccess, TicketPriority, SupportPlan, Signature, BusinessCalendar, Sla, SlaTarget
Tickets	Ticket, Article, Attachment, TicketTag, TicketLink, TicketHistory, Mention, Subscription, SharedDraft, ChecklistItem, TimeEntry
Reglas y productividad	Trigger, Scheduler, Overview, CoreWorkflow, Macro, TextModule, TicketTemplate, ObjectAttribute
Canales	EmailChannel, PostmasterFilter, InboundMessage, CallLog, MessagingChannel, ChatSession
Privacidad y sistema	AuditLog, Notification, DataPrivacyTask, RetentionPolicy, AppSetting
Integraciones	Webhook, WebhookDelivery, DevOpsLink, TeamsChannel
Base de conocimiento	KbCategory, KbAnswer, KbTranslation, KbTranslationVersion

Roles de base de datos

- `app_migrator`: dueño del esquema; solo lo usa el contenedor `migrator`.
- `app_rw`: la aplicación; solo SELECT, INSERT, UPDATE y DELETE, máximo 40 conexiones.
- Conexiones solo por TLS (`hostssl`) con `scram-sha-256`.

Migraciones

Prisma Migrate, aplicadas por el desplegador antes de levantar la nueva versión. Política expand/contract: una reversión de imágenes no deshace el esquema, así que cada migración debe ser compatible con la versión anterior (ADR-0008).

Redis

Sesiones (solo hashes), límites de frecuencia, valores de un solo uso (TOTP, OIDC, WebAuthn, CTI), colas BullMQ, pub/sub de eventos y presencia. Persistencia AOF cada segundo.

07 Infraestructura en OCI

Todo se crea con Terraform (`infra/terraform`, proveedor oracle/oci 9.x).

RECURSO	CONFIGURACIÓN
Región y compartimento	`mx-queretaro-1` (la validación solo admite regiones de México); compartimento `mesa-soporte-prod`.
Instancia	`mesa-prod-app`: VM.Standard.A1.Flex, 2 OCPU, 12 GB, Ubuntu 24.04 aarch64, solo IMDSv2, red paravirtualizada.
Discos	Arranque 50 GB; datos 100 GB en `/srv/mesa` (XFS, `prevent_destroy`), ambos cifrados con la llave maestra.
Vault	`mesa-prod-vault` con llave `mesa-prod-master-key` (AES-256 en HSM). Terraform nunca crea valores de secretos.
IAM	Grupo dinámico `mesa-prod-vm` (instance principal) con permisos mínimos; usuario de servicio de pgBackRest solo para `respaldos-pitr`; grupos humanos de infraestructura y auditoría.
Seguridad	Cloud Guard, Security Zone (Maximum Security), escaneo semanal de vulnerabilidades del host.
Observabilidad	Topic `mesa-prod-alerts`; alarmas de CPU, memoria, disco, respaldo fallido, falta de latido y aplicación no sana; logs del host y de `/var/log/mesa` por 90 días.
Estado de Terraform	Backend OCI, bucket `mesa-terraform-state`, clave `mesa-soporte/prod.tfstate`.

Buckets

BUCKET	USO	RETENCIÓN
`adjuntos`	Archivos de tickets	Versionado; versiones previas se borran a 14 días
`respaldos-pitr`	Repositorio de pgBackRest	Versionado; versiones previas se borran a 14 días
`respaldos-inmutables`	Volcados diarios cifrados	Regla WORM de 30 días (se bloquea a mano tras la primera prueba de restauración); borrado a 90 días

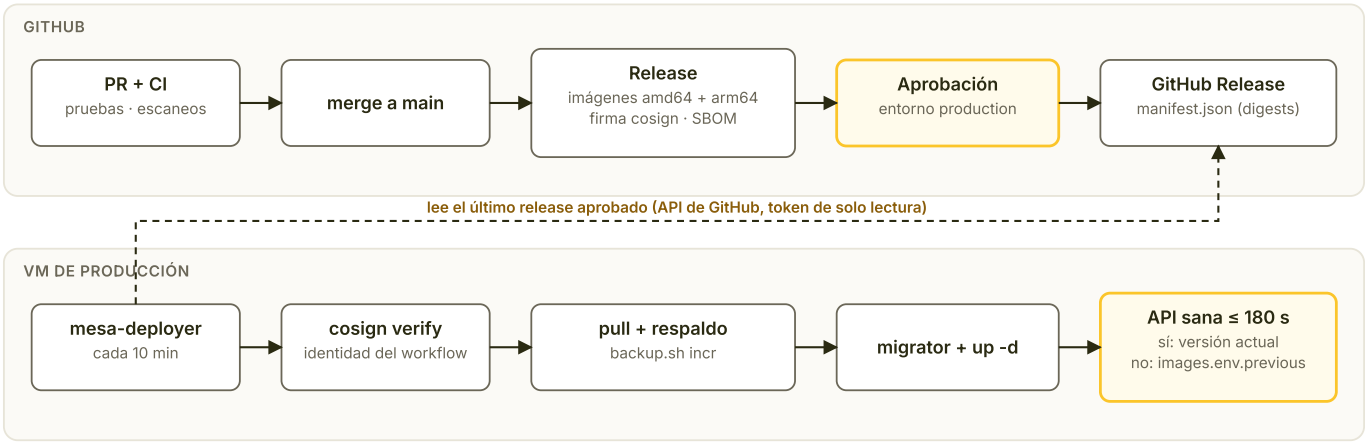
Arranque de la VM (cloud-init)

Instala Docker y Compose, age, jq, auditd, actualizaciones automáticas, OCI CLI y cosign verificado; endurece sshd, sysctl y Docker; prepara el tmpfs de secretos y el volumen de datos; y deja en `/opt/mesa` el compose, los scripts y las unidades systemd. Los cambios de cloud-init requieren reemplazar la instancia (`terraform plan -replace=oci_core_instance.app`), porque `metadata` está en `ignore_changes`.

> **CAPACIDAD DE A1**
> Si OCI responde «Out of host capacity» al crear la instancia, se reintenta el `apply` en un ciclo hasta que haya capacidad.

08 Integración y despliegue

Del pull request a producción sin que GitHub tenga acceso al servidor.



De un PR a producción. GitHub nunca entra al servidor: la VM *jala* el release aprobado y solo acepta imágenes firmadas por el workflow de release en `main`.

CI (en cada PR y en main)

JOB	QUÉ VALIDA
Lint, types, unit tests, build	ESLint, Prettier, TypeScript, Vitest y build.
Integration and browser tests	PostgreSQL y Redis reales, almacenamiento S3 y correo de prueba; pruebas de integración y Playwright.
Backup/PITR, deployer, secrets...	shellcheck, pruebas del desplegador y de secretos, respaldo y PITR reales, smoke de la pila de producción con OWASP ZAP.
Secrets, SAST, dependencies, IaC...	gitleaks, Semgrep, `pnpm audit`, Trivy sobre IaC e imágenes.
Terraform	`fmt`, `validate` y tflint.
linux/arm64 build and smoke test	Solo en main: build y smoke en arm64 con QEMU.

Release

En cada push a main: reutiliza la CI, construye las cinco imágenes (api, migrator, worker, web, postgres) para amd64 y arm64 con SBOM y procedencia, las publica en GHCR y las firma con cosign keyless. Tras la aprobación del entorno `production` publica el release `mesa-vAAAA.MM.DD-N` con `manifest.json`.

Desplegador en la VM

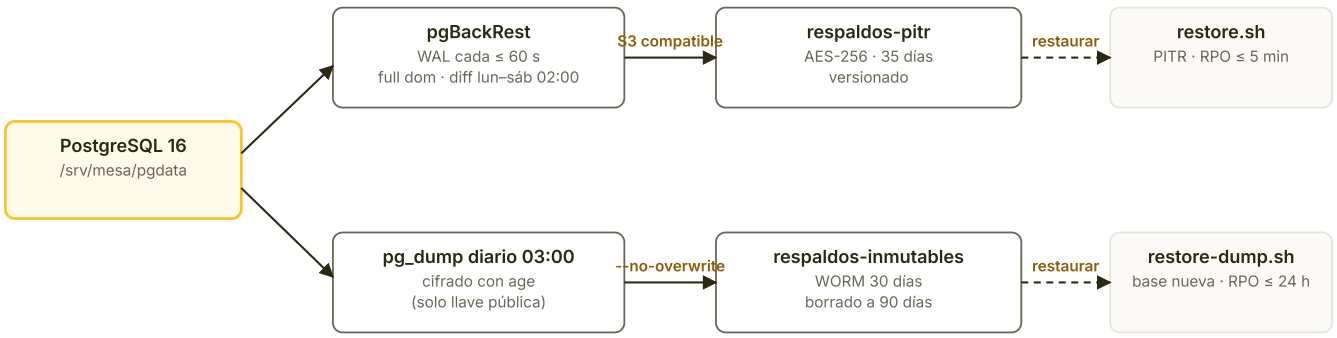
`mesa-deployer.timer` corre cada 10 minutos. Usa `github_read_token` (lectura del repositorio) y `ghcr_read_token` (`read:packages`). Solo acepta imágenes fijadas por digest y firmadas por el workflow de release en `main`. El estado queda en `/var/lib/mesa/deploy/` y el log en `/var/log/mesa/deployer.log`.

Unidades systemd

UNIDAD	CUÁNDO	QUÉ HACE
`mesa-secrets-sync`	al arrancar	Copia los secretos de Vault al tmpfs.
`mesa-stack`	al arrancar	Levanta la pila con `stack.env` e `images.env`.
`mesa-deployer.timer`	cada 10 min	Instala el último release aprobado.
`mesa-backup-full.timer`	domingo 02:00	Respaldo completo de pgBackRest.
`mesa-backup-diff.timer`	lun–sáb 02:00	Respaldo diferencial.
`mesa-backup-check.timer`	cada hora	Verifica el archivado de WAL.
`mesa-dump.timer`	diario 03:00	Volcado cifrado al bucket inmutable.
`mesa-report.timer`	cada 5 min	Métricas: latido, disco, salud de la aplicación.
`mesa-alert@`	al fallar otra unidad	Publica la alerta en el topic de OCI.

09 Respaldos y recuperación

Objetivos: RPO ≤ 5 min (≤ 24 h con el volcado inmutable) y RTO ≤ 4 h.



Respaldos. Dos caminos independientes: pgBackRest para recuperar a cualquier minuto (PITR) y un volcado diario cifrado con una llave cuya parte privada nunca está en el servidor.

ESCENARIO	PROCEDIMIENTO
A. Error humano	PITR con `restore.sh --time "<fecha>"`; los datos actuales se mueven a `/srv/mesa/pgdata.before-restore-<fecha>`, nunca se borran.
B. Pérdida del servidor	`terraform apply`, secretos, desplegador y `restore.sh --latest --yes`.
C. Servidor comprometido	Restaurar el volcado inmutable desde un equipo limpio con `restore-dump.sh` a una base nueva.
D. Prueba trimestral	Obligatoria; también se prueba en CI en cada cambio.

LLAVE DE AGE

El servidor solo tiene la llave **pública** (`/opt/mesa/backup-age-recipient.txt`). La privada vive en el gestor de contraseñas y en un sobre sellado; sin ella el volcado inmutable no se puede abrir.

Detalle completo: `docs/runbooks/restauracion.md` y ADR-0009.

10 Seguridad

Controles transversales y decisiones de arquitectura.

Secretos

1. `infra/scripts/bootstrap-secrets.sh` genera los valores aleatorios, pide los externos y los guarda en OCI Vault; produce `secrets.map` (nombres y OCID, sin valores).
2. `secrets.map` se instala en `/opt/mesa/secrets.map`.
3. `mesa-secrets-sync` lee Vault con el instance principal y escribe archivos 0400 en el tmpfs `/run/mesa/secrets`.
4. Un secreto con valor `unused` deja su integración desactivada. Rotación: `docs/runbooks/rotacion-secretos.md`.

Cifrado

- **En reposo**: llave HSM para volúmenes y buckets; pgBackRest cifrado; age en volcados; AES-256-GCM por campo para semillas TOTP, URLs de Teams y similares (con rotación por id de llave).
- **En tránsito**: Cloudflare y túnel; PostgreSQL con TLS dentro de Docker; SMTP con STARTTLS. El cifrado en tránsito del volumen no está disponible en A1.

Navegador

CSP estricta (`script-src 'self'`, sin estilos ni scripts en línea, `frame-ancestors 'none'`), HSTS de 2 años, `nosniff`, Permissions-Policy restrictiva, COOP y CORP `same-origin`.

Gobierno

- Modelo de amenazas STRIDE con 55 amenazas, mitigación y prueba: `docs/seguridad/modelo-amenazas.md`.
- OWASP ASVS 4.0 nivel 2: `docs/seguridad/matriz-asvs.md`.
- Respuesta a incidentes (triage ≤ 1 h, LFPDPPP): `docs/seguridad/respuesta-incidentes.md`.

- Riesgos aceptados: un solo servidor sin alta disponibilidad, certificado autofirmado de PostgreSQL interno, llave de larga duración de pgBackRest, detectores por patrones (datos sensibles e IA), `img-src https:` en la CSP.

Decisiones de arquitectura (ADR)

ADR	TEMA
0001	Stack tecnológico
0002	Autorización «denegar por defecto»
0003	Sesiones en servidor, cookies por host y CSRF
0004	Almacenamiento de archivos
0005	Secretos como archivos desde OCI Vault
0006	Correo saliente por cola
0007	Infraestructura en OCI México
0008	Despliegue pull con firmas verificadas
0009	Respaldos y recuperación
0010	Bases efímeras para pruebas
0011	Reglas como datos
0012	Canal de correo
0013	Adjuntos, antivirus y datos sensibles
0014	Tiempo real, tokens de API y ver como cliente
0015	Base de conocimiento bilingüe
0016	Reportes y exportación
0017	Azure DevOps, Teams y webhooks
0018	SSO OIDC y WebAuthn
0019	API de Inncentives y widget
0020	Telefonía, WhatsApp y SMS

ADR	TEMA
0021	Chat en vivo
0022	IA para agentes

11 Operación

Archivos, comandos y verificaciones del día a día en el servidor.

Archivos en la VM

RUTA	CONTENIDO
`/opt/mesa/stack.env`	Hosts, región y namespace de OCI, remitente de correo y opciones de Graph.
`/opt/mesa/images.env`	Imágenes por digest de la versión actual (y `.previous`).
`/opt/mesa/deployer.env`	Repositorio, rutas y topic de alertas del desplegador.
`/opt/mesa/secrets.map`	Nombres y OCID de los secretos.
`/opt/mesa/scripts/`	Desplegador, respaldos, restauración, secretos y métricas.
`/run/mesa/secrets`	Secretos en memoria (tmpfs).
`/srv/mesa/`	Volumen de datos: `pgdata`, `redis`, `clamav`.
`/var/lib/mesa/deploy/`	Versión actual e historial de despliegues.
`/var/log/mesa/`	Logs del desplegador, respaldos y secretos.

Comandos frecuentes

PARA	COMANDO
Ver contenedores	`sudo docker ps --format '{{{{.Names}}}} {{{{.Status}}}}'`
Logs de un servicio	`cd /opt/mesa && sudo docker compose -p mesa -f docker-compose.prod.yml --env-file stack.env --env-file images.env logs -f api`
Reiniciar la pila	`sudo systemctl restart mesa-stack`
Desplegar ahora	`sudo systemctl start mesa-deployer` (o `deployer.sh --force`)
Versión instalada	`cat /var/lib/mesa/deploy/current-version`
Respaldos	`sudo /opt/mesa/scripts/backup.sh full|diff|incr|check|info`
Temporizadores	`systemctl list-timers 'mesa-*'` (deben aparecer 6)
Primer administrador	`… run --rm api dist/scripts/bootstrap-admin.js <correo> "<Nombre>"`

Salud

`GET /health` (proceso vivo) y `GET /ready` (base de datos y Redis), sin prefijo `/api/v1`. `mesa-report` publica `app_healthy` y el latido cada 5 minutos; las alarmas avisan al topic `mesa-prod-alerts`.

Runbooks

- `docs/despliegue.md`: aprovisionamiento completo.
- `docs/runbooks/restauracion.md`: restauración de PostgreSQL.

- `docs/runbooks/rotacion-secretos.md`: rotación (tokens de GitHub y llave de pgBackRest cada 90 días; SMTP 180; resto anual).
- `docs/seguridad/respuesta-incidentes.md`.

12 Estado de producción

Lo que difiere de la guía de despliegue y lo que queda pendiente, al 20 de octubre de 2026.

Diferencias con la guía de despliegue

- Hosts en `neitek.net` con DNS en DigitalOcean y Cloudflare for SaaS (dominio técnico `neiteksupport.com`), en lugar de una zona propia en Cloudflare.
- Acceso de operadores por port forwarding de Bastion con llaves en `admin_ssh_public_keys`; la security list lleva la salida TCP 22 hacia la subred.
- El aprovisionamiento se hizo desde Windows (OCI CLI en un venv, Git Bash para `bootstrap-secrets.sh`).

Pendientes

- Actualizar `docs/despliegue.md` con las diferencias anteriores.
- Configurar el correo de Microsoft 365 (envío del sistema y buzones de entrada). Mientras tanto, las invitaciones se generan con un token manual desde el servidor.
- Completar en Cloudflare el WAF, el límite de autenticación y Cloudflare Access para el host de agentes.
- Configuración inicial de la aplicación (§9 bis de la guía).
- Bloquear la regla WORM de `respaldos-inmutables` después de la primera prueba de restauración.

Documento interno de Neitek. Fuente: el repositorio `mesa-soporte` (docs, ADRs, Terraform, compose y scripts) y el registro de la puesta en marcha. Si algo cambia, actualiza `docs/manuales/build_tecnico.py`.